

1. Question

An organization is maintaining the data of employees according to cadre of employee with the following parameters:

Name

ID

Designation

Salary

Promotion Status

Apply constructor overloading to implement it.

2. Steps for Execution

1.Install Java JDK on the system.

2.Open Eclipse, IntelliJ IDEA, NetBeans, VS Code, or any Java IDE.

3.Create a Java file named:

- Employee.java
- Enter the given Java program.
- Save the program.

4.Compile the program:

```
javac Employee.java
```

5.Run the program:

```
java Employee
```

6.The program creates three employee objects using different constructors.

7.The first object uses the default constructor.

8.The second object uses a two-parameter constructor.

9.The third object uses a five-parameter constructor.

10.The display() method displays the details of each employee.

11.The program demonstrates constructor overloading because the same class contains multiple constructors with different parameters.

3. Algorithm

Step 1: Start the program.

Step 2: Create a class Employee1.

Step 3: Declare employee data members:

name

id

designation

salary

promotionStatus

Step 4: Create a default constructor with no parameters.

Assign default values:

Name = Unknown

ID = 0

Designation = Not Assigned

Salary = 0

Promotion Status = Not Decided

Step 5: Create a two-parameter constructor accepting:

name

id

Assign default values for the remaining employee details.

Step 6: Create a five-parameter constructor accepting:

name

id

designation

salary

promotionStatus

Step 7: Create a display() method to display employee details.

Step 8: Create the first employee object using the default constructor.

```
e1 = new Employee1()
```

Step 9: Create the second employee object using the two-parameter constructor.

```
e2 = new Employee1("Ravi", 102)
```

Step 10: Create the third employee object using the five-parameter constructor.

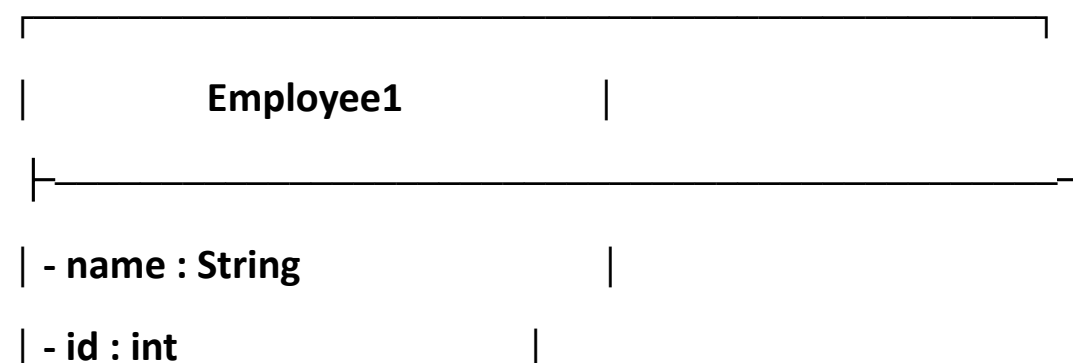
```
e3 = new Employee1(  
    "Anil", 112, "Manager", 10000, "Promoted"  
)
```

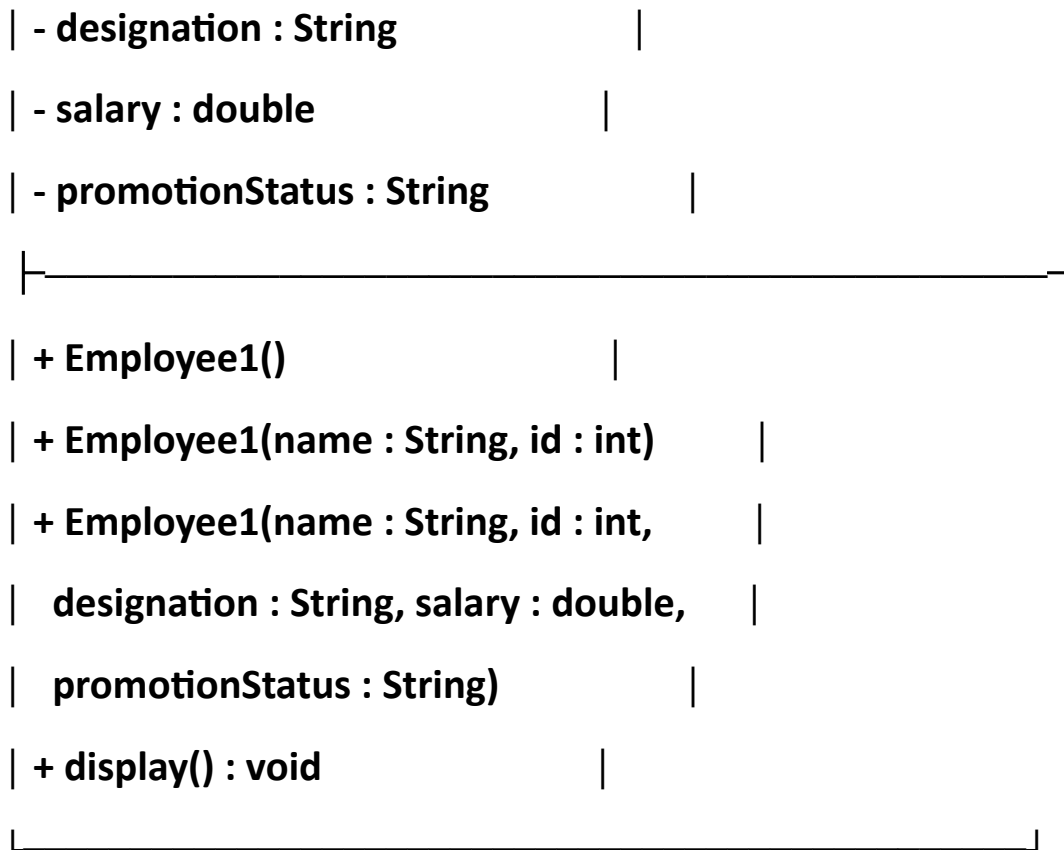
Step 11: Call the display() method for all three objects.

Step 12: Display the employee details.

Step 13: Stop the program.

UML CLASS DIAGRAM





CODE:

```
package javacore;

class Employee1 {
    String name;
    int id;
    String designation;
    double salary;
    String promotionStatus;
    public Employee1() {
        name = "Unknown";
        id = 0;
        designation = "Not Assigned";
    }
}
```

```
        salary = 0;

        promotionStatus = "Not Decided";
    }

    public Employee1(String name, int id) {

        this.name = name;

        this.id = id;

        designation = "Not Assigned";

        salary = 0;

        promotionStatus = "Not Decided";
    }

    public Employee1(String name, int id, String designation,
        double salary, String promotionStatus) {

        this.name = name;

        this.id = id;

        this.designation = designation;

        this.salary = salary;

        this.promotionStatus = promotionStatus;
    }

    void display() {

        System.out.println("Name: " + name);

        System.out.println("ID: " + id);

        System.out.println("Designation: " + designation);

        System.out.println("Salary: " + salary);

        System.out.println("Promotion Status: " + promotionStatus);

        System.out.println();
    }
}
```

```
    }  
}  
  
public class Employee {  
    public static void main(String[] args) {  
        Employee1 e1 = new Employee1();  
        Employee1 e2 = new Employee1("Ravi", 102);  
        Employee1 e3 = new Employee1("Anil", 112, "Manager", 10000,  
"Promoted");  
        e1.display();  
        e2.display();  
        e3.display();  
    }  
}
```

OUTPUT SCREENSHOT (SAMPLE RUN):

Name: Unknown

ID: 0

Designation: Not Assigned

Salary: 0.0

Promotion Status: Not Decided

Name: Ravi

ID: 102

Designation: Not Assigned

Salary: 0.0

Promotion Status: Not Decided

Name: Anil

ID: 112

Designation: Manager

Salary: 10000.0

Promotion Status: Promoted